Event Last call on early bird discount! Get your ticket to MongoDB.local London. Register now >>

Generating Globally Unique Identifiers for Use with MongoDB

Get Started with MongoDB Atlas

Alexander Marquardt
February 14, 2017 | Updated: March 5, 2021
#Technical

By default, MongoDB generates a unique ObjectID identifier that is assigned to the _id field in a new document before writing that document to the database. In many cases the default unique identifiers assigned by MongoDB will meet application requirements. However, in some cases an application may need to create custom unique identifiers, such as:

- The application may require unique identifiers with a precise number of digits. For example, unique 12 digit identifiers might be required for bank account or credit card numbers.
- Unique identifiers may need to be generated in a monotonically increasing and continuous sequential order.
- Unique identifiers may need to be independent of a specific database vendor.

Due to the multi-threaded and distributed nature of modern applications, it is not always a straightforward task to generate unique identifiers that satisfy application requirements.

Overview

This posting covers the following topics:

- Ensure identifier uniqueness at the database level
- Use ObjectID as a unique identifier
- Use a single counter document to generate unique identifiers one at a time
- Use a single counter document that allocates batches of unique identifiers
- Use multiple counter documents that allocate batches of unique identifiers
- Randomly choose a unique identifier in the application and retry if it is already assigned
- Use a standard UUID algorithm for application-level unique identifier generation

Ensure identifier uniqueness at the database level

All of the approaches that we propose in this post will generate identifiers that are globally unique for all practical purposes, however depending on the chosen approach there may be remote edge cases where the generated identifier may not be absolutely globally unique. If the risk of a clash of unique identifiers is deemed to be sufficiently remote or the consequences of such a clash are minor, then one may consider writing to the database without additional uniqueness checking. This is a valid approach that may be adopted by some applications.

However, if it is absolutely essential to enforce that the generated identifier is globally unique, then code may be written defensively to guarantee that the database will never write the same unique identifier twice. This is relatively easy to implement on a non-sharded collection by specifying a unique index on a particular field, which prevents a duplicate value for that field from ever being written to the collection. It is also worth noting that the _id field has a unique index on it, so values assigned to _id will be verified for uniqueness by default.

If a document fails to be written due to a collision on a field that has a unique index, then the application should (1) catch this error, (2) generate and assign a new unique identifier to the document, and (3) try to write the document to the database again.

If a collection is sharded, there are restrictions on the use of unique indexes. If the restrictions for using a unique index on a sharded collection cannot be satisfied, then a proxy collection may be used to guarantee uniqueness before writing data to the database.

Use ObjectID as a unique identifier

Description

MongoDB database drivers by default generate an ObjectID identifier that is assigned to the _id field of each document. In many cases the ObjectID may be used as a unique identifier in an application.

ObjectID is a 96-bit number which is composed as follows:

- a 4-byte *timestamp value* representing the seconds since the Unix epoch (which will not run out of seconds until the year 2106)
- a 5-byte *random value*, and
- a 3-byte *incrementing counter*, starting with a random value.

Benefits

- ObjectID is automatically generated by the database drivers, and will be assigned to the _id field of each document.
- ObjectID can be considered globally unique for all practical purposes.
- ObjectID encodes the timestamp of its creation time, which may be used for queries or to sort by creation time.
- ObjectID is mostly monotonically increasing.
- ObjectID is 96-bits, which is smaller than some (eg. 128-bit) alternative UUID implementations, which will result in slightly smaller documents that will use slightly less disk space and RAM than these alternatives.

Drawbacks

- Some businesses may be reluctant to link their application logic to an identifier that is generated by a specific database product.
- ObjectID can be considered globally unique for all practical purposes, but there are edge cases where it may not be truly globally unique.
- At 96 bits, the ObjectId is longer than some (e.g. 64-bit) alternative solutions, which means that documents will be slightly larger and will require slightly more disk space and RAM than these alternatives.

Use a single counter document to generate unique identifiers one at a time

Disclaimer

The approach described in this section is generally not recommended due to the potential for the single counter document to become a bottleneck in the application.

Description

A unique identifier may be required in a monotonically increasing and continuous sequential order, which is similar to the sequence functionality that is implemented by some RDBMSs.

This may be achieved by implementing a solution that is based on a centralised counter document that resides in a collection that we have called uniqueIdentifierCounter. This centralised counter document is the only document in the uniqueIdentifierCounter collection, and its COUNT field will track the current unique identifier value.

Each time a new unique identifier is needed, findAndModify will be used on the counter document to atomically increment the COUNT field and return the pre-increment document. The COUNT value can then be used by the application as a unique identifier. For example the application could assign the COUNT value to the _id field of a document that will be written to a given collection.

Example implementation

A counter document for unique identifier generation could look as follows:

```
{
    "_id"   : "UNIQUE COUNT DOCUMENT IDENTIFIER",
    "COUNT" : 0,
    "NOTES" : “Increment COUNT using findAndModify to ensure that the COUNT field will b
}
```

And the unique identifier-generation document could be atomically requested and incremented as follows. Note that by default the document returned from findAndModify is the pre-modification document:

```
db.uniqueIdentifierCounter.findAndModify({
    query: { _id: "UNIQUE COUNT DOCUMENT IDENTIFIER" },
    update: {
        $inc: { COUNT: 1 },
    },
    writeConcern: 'majority'
})
```

Benefits

- Easy to implement.
- Unique identifiers are generated in a continuous and monotonically increasing manner.
- It is possible to specify the type of value that is stored in the COUNT field, for example 32-bits (int) or 64-bits (long), to give some control over how the COUNT field will impact document size.

Drawbacks

- This approach will likely generate a serious bottleneck in the system, as there will be contention caused by many threads simultaneously accessing the single counter document.
- Depending on replication lag and time to flush the counter document to disk, this technique will limit the speed of unique identifier generation. If we assume that it takes 25ms for the counter document to be persisted and replicated to the database then this method would only be able to generate 40 new unique identifiers per second. If the application is waiting for unique identifier values before new documents can be inserted into a given collection, then these inserts will have a maximum write speed of 40 documents per second. Without such a bottleneck, we would expect a well functioning database to be able to write tens of thousands of documents per second.

Use a single counter document that allocates batches of unique identifiers

Description

This approach is similar to the previous approach, with the difference being that instead of incrementing the COUNT value by 1, we may wish to increment it by a larger number that will represent a batch of unique identifiers that will be allocated by the database to the application.

For example, if the application knows that it needs 1000 new unique identifiers, then the application would use findAndModify() to atomically get the current COUNT and increment the COUNT value by 1000. The document returned from the findAndModify command would contain the starting value for the batch of unique identifiers, and the application would loop over 1000 values from that starting point.

Note that with this approach an application may pass in whatever value it wishes for incrementing the COUNT value, and therefore this approach can be made to be flexible depending on the application's requirements. For large batches this increment would be a large number, and for a single identifier this would be set to 1.

Example implementation

The following demonstrates the javascript shell commands that would atomically increment the COUNT by 1000 and return the previous (before the increment) counter document:

```
var seq_increment = 1000;
db.uniqueIdentifierCounter.findAndModify({
    query: { _id: "UNIQUE COUNT DOCUMENT IDENTIFIER" },
    update: {
        $inc: {COUNT: seq_increment },
    }
    writeConcern: 'majority'
})
```

Benefits

- Relatively easy to implement.
- Unique identifiers are generated by the database in a monotonically increasing manner, and will likely be used by the application in a mostly monotonically increasing manner.
- This approach has the potential to dramatically reduce the number of requests and updates to the counter document, which may eliminate the bottleneck described in the previous approach.

Drawbacks

- There is potential for bottlenecks if the application requests small unique identifier ranges with each request.
- The application must understand that the number received from the database is meant as a range of values.
- Multiple threads requesting batches of unique identifiers at a similar moment in time could cause the allocated identifiers to be used by the application in an order that is not strictly monotonically increasing, as each thread takes time to work through its allocated batch.
- If the application misbehaves, then it could burn through a large number of unique identifiers without actually using them.

Use multiple counter documents that allocate batches of unique identifiers

Description

This approach is similar to the previous approach, but instead of having a single counter document, we could have many counter documents stored in the uniqueIdentifierCounter collection.

For example, there may be 1000 counter documents (numbered 0 to 999) each responsible for allocating 1 billion unique numbers that are taken from a specific range that has been allocated to each counter. In this example, counter 499 would be responsible for allocating values from 499000000000 to 499999999999. Note that this particular example results in unique numbers ranging from 0 to 999,999,999,999 which is a 12 digit number.

Example implementation

Below we show the format and initial values assigned to counter documents in the uniqueIdentifierCounter collection:

```
/* The following would be the initial state of the 0th counter document, which is respon
{
    "_id"  : "COUNTER DOCUMENT NUMBER 0",
    "MAX_VALUE": 999999999,
    "COUNT"  : 0,
    "NOTES" : "Increment COUNT using findAndModify to ensure that the COUNT field will b
}

/* The following would be the initial state of 499th counter document, which is responsi
{
    "_id"  : "COUNTER DOCUMENT NUMBER 499"),
    "MAX_VALUE": 499999999999,
    "COUNT"  : 499000000000,
    "NOTES" : "Increment COUNT using findAndModify to ensure that the COUNT field will b
}
/* Etc… */
```

With this approach, each time the application needs a new unique number or a range of unique numbers, the application could randomly generate a number between 0 and 999 which it would use to perform a query against the _id attribute in the uniqueIdentifierCounter collection. This would select a particular counter document from which to retrieve the unique identifier batch.

This is demonstrated in the following example, in which we randomly select one of the counter documents, and request a batch of 100 unique numbers from that counter:

```
var which_counter_to_query = Math.floor((Math.random()*1000));
var seq_increment = 100;
db.Unique Identifier_counter.findAndModify({
    query: { _id:  "COUNTER DOCUMENT NUMBER " + which_counter_to_query},
    update: {
        $inc: {COUNT: seq_increment },
    },
    writeConcern: 'majority'
})
```

Benefits

- Compared to the previous approaches, this approach will reduce contention by having fewer threads simultaneously accessing each counter document.

Drawbacks

- This is more complicated than the previous implementations.
- There is potential for bottlenecks if this approach is used for generating small batches and has a small number of counter documents.
- The number of counter documents must be predefined and the range of unique identifiers assigned to each counter document needs to be allocated in advance.
- Care must be taken to ensure that the pre-defined range that is assigned to each counter document does not roll-over.

Randomly choose a unique identifier in the application and retry if it is already assigned

Description

This approach relies on the fact that if a unique index is defined in a collection, that any document written to that collection must have a unique value assigned to that field in order for it to be successfully written. Therefore, we can randomly generate a unique identifier number in the application, assign it to the unique field in a document, and try to write that document to the collection. If the write succeeds, then we know that the value that we assigned to it is unique. If the write fails, then the application must catch the failure and randomly generate a new unique identifier which can then be used to write the document to the collection. If the number of collisions is low, then this can be an efficient way to write documents to the database with each document having a guaranteed unique identifier.

Example implementation

If we know that we will have a maximum of one billion records in our database, in order to have a low probability of selecting an identifier that has already been assigned (aka a collision), we could randomly choose a number between 0 and 999,999,999,999 (from zero to one trillion minus one). For this example, the range that we are using to select the random number is 1000 times bigger than the number of documents that we expect to write to this collection, which results in a worst case expected 0.1% chance of a collision.

We would then assign the randomly generated number to a field that has a unique index, and write that document to the database. If the write succeeds, then we know that the identifier is unique. If the write fails, then the application must randomly choose another identifier and try again until the write succeeds. Note that there are some restrictions on the use of unique indexes when used in sharded clusters. If the restrictions for using a unique index on a sharded collection cannot be satisfied, then a proxy collection may be used to help generate unique indexes.

Benefits

This approach is not difficult to implement.

Drawbacks

- Some businesses do not like the random nature of this approach to unique identifier generation.
- Testing unique identifier collisions may be tricky since it relies on random numbers.
- If the random number generator is not good then there may be a potential for multiple collisions and multiple retries required before successfully writing a document.

Use a standard UUID algorithm for application-level unique identifier generation

Description

RFC 4122 defines a standard for generating 128-bit UUIDs. The RFC specifies different algorithms for generating UUIDs, which are called RFC 4122 versions.

Standard libraries exist that will generate 128-bit UUIDs at the application level. The BSON specification defines UUID as a valid subtype, and can be found at: http://bsonspec.org/spec.html.

Benefits

- This approach effectively addresses UUID-related bottlenecks.
- UUIDs are fully generated in application code, which saves a round-trip to the database that is required with some of the alternative approaches.
- This is a standard implementation that relies on existing libraries.
- This approach does not use a database-generated identifier for business functions.
- For all practical purposes, UUIDs generated with this approach are globally unique.
- If RFC 4122 version 1 or version 2 is used and is properly implemented, then the generated UUIDs are guaranteed to be globally unique.

Drawbacks

- These unique identifiers are 128-bits, which is longer than some alternative approaches. This results in slightly larger documents, and therefore uses slightly more disk space and memory.
- For RFC 4122 algorithm versions other than version 1 and version 2, it is theoretically possible to generate a UUID that is not absolutely globally unique. However, for all practical purposes the resulting UUIDs are globally unique. More information can be found in the Wikipedia article about UUID generation.

About the Author

Alexander is a Senior Consulting Engineer at MongoDB who has worked closely with clients of all shapes and sizes, advising on all aspects of running MongoDB at scale and in mission-critical environments. Prior to joining MongoDB, Alexander was the founder and principal developer at Lexabit Inc., which owned several social networking websites that were scaled out to tens of thousands of unique visitors per day. Alex has a Masters Degree in Computer Engineering from the University of Toronto, a B.Sc. from the University of Manitoba, and an MBA from the IESE Business School.

Get Started with MongoDB Atlas

Run MongoDB in the cloud for free with MongoDB Atlas. No credit card required.

Get Started Free!

← Previous

Leaf in the Wild: Powering Smart Factory IoT with MongoDB

BEET Analytics OEMs MongoDB for its Envision manufacturing IOT platform helping Envision deliver 1-2 orders of...

February 13, 2017

Next →

AI Agents, Hybrid Search, and Indexing with LangChain and MongoDB

Since we announced integration with LangChain last year, MongoDB has been building out tooling to help...

September 12, 2024

© 2024 MongoDB, Inc.